National University of Computer and Emerging Sciences

Agentic AI CS-4015 (Section: A, B, C)

Course Project PHASE 1

Due Date: 9th April 2026, Thursday

THE WRITER'S ROOM:

Autonomous Story and Image Generation Layer

■ Introduction

Phase 1 establishes the **foundation of the PROJECT MONTAGE system**, where raw human intent is transformed into a **structured, machine-interpretable narrative representation.**

Unlike traditional pipelines, this module operates as a **multi-agent creative system**, where specialized agents:

- Collaborate autonomously
- Maintain shared memory
- Operate under **MCP-based tool discovery constraints**
- Are orchestrated using **LangGraph stateful workflows**

This phase directly satisfies the **core requirement** of transforming raw human intent into structured multi-scene scripts with visual cues, enabling downstream multimodal synthesis.

■ System Objectives

The primary goal of phase 1 is to produce:

- *scene_manifest.json* -> structured screenplay
- *character_db.json* -> persistent character identity store
- *image_assets/* -> AI-generated character visuals

Additionally, this phase ensures:

- Compatibility with **downstream video/audio agents**
- Support for both:
  - Manual script input
  - Autonomous LLM generation

■ Architectural Design Principles

3.1 Multi-Agent Collaboration Model

This phase follows a **Supervisor-Worker hierarchical model**

- Supervisor (implicit via LangGraph routing)
- Worker Agents:
  - Scriptwriter
  - Character Designer
  - Image Synthesis
  - Validator

Each agent:

- Has **clearly scoped responsibilities**
- Interacts via **shared state**
- Uses **MCP tools dynamically (no hardcoding)**

3.2 MCP-based Tool Discovery Constraint

A strict constraint enforced:

! All tools must be discovered dynamically via MCP – no hardcoded APIs.

Thus:

- Agents query MCP registry at runtime
- Tools are invoked via structured schemas

Example:

```JSON
{
  "tool": "generate_script_segment",
  "input": {
    "prompt": "...",
    "num_scenes": 5
  }
}
```

3.3 Stateful Memory System

All agents interact with a **persistent memory layer:**

- Vector DB (ChromaDB / FAISS)
- Stores:
  - Script history
  - Character metadata

- Image references

This directly supports:

- Agent continuity
- Recovery from failure
- Future personalization

■ Script Intake Logic

This phase implements **dual-mode script ingestion**

✓ **Mode 1: Direct Script Injection (Manual)**

Workflow:

1. User uploads script
2. Script Validator Agent:
    a. Check structure:
        i. Scene headings
        ii. Dialogue labels
        iii. Action descriptions
3. Converts script -> standardized JSON

✓ **Mode 2: Autonomous Script Generation**

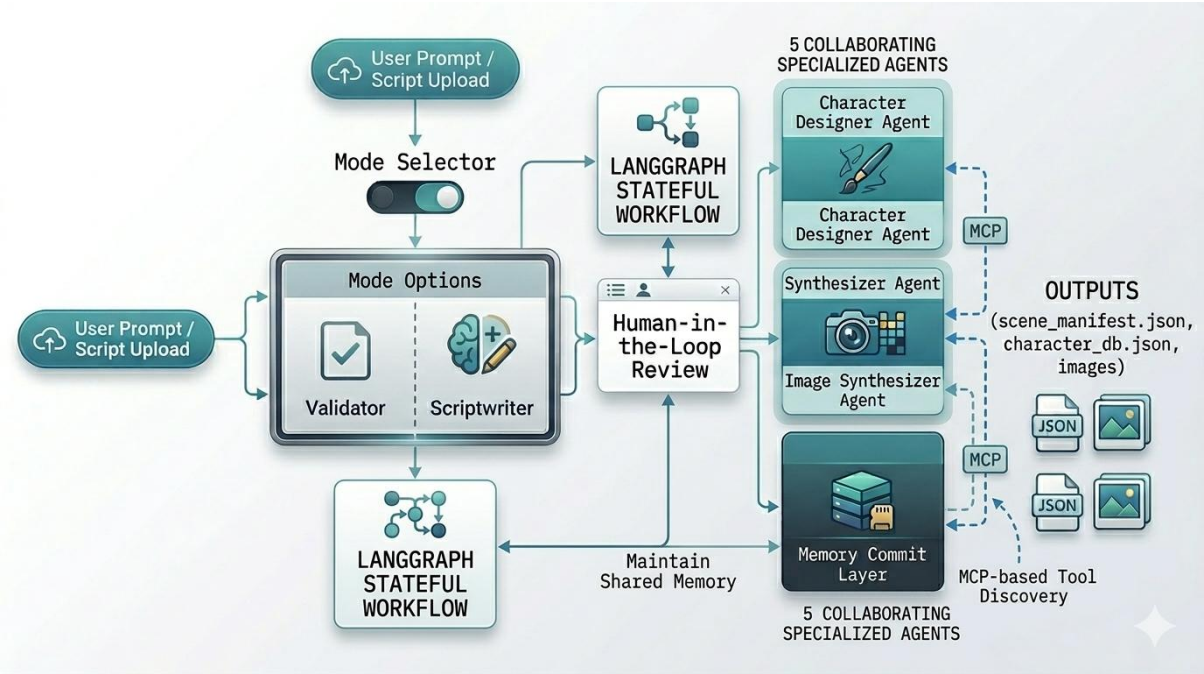
Workflow:

1. User provides prompt
2. Scriptwriter Agent:
    a. Expands into:
        i. Multi-scene screenplay
        ii. Dialogues
        iii. Visual Cues

✓ **Output Format (Standardized JSON)**

```JSON
{
  "scenes": [
    {
      "scene_id": 1,
      "location": "City street",
      "characters": ["A", "B"],
      "dialogue": [
        {
          "speaker": "A",
          "line": "We must act now",
          "visual_cue": "Close-up, tense lighting"
        }
      ]
    }
  ]
}
```

■ System Architecture (High-Level Flow)



■ Agents Definitions

6.1 Scriptwriter Agent

Role:

Transform abstract prompts into structured, production-ready scripts.

Core Responsibilities:

- Scene segmentation
- Dialogue generation
- Visual cue injection
- Character consistency enforcement

Reasoning Loop:

- Interpret prompt
- Decompose into scenes
- Generate dialogue
- Attach visual context

MCP Tools:

- *generate_script_segment*
- *commit_memory*

6.2 Script Validator Agent

Role:

Ensures correctness of manually provided scripts.

Validation Checks:

- Scene headers exist
- Dialogue is labeled
- Actions are structured

Failure Handling:

- Reject script
- Suggest corrections

6.3 Human-in-the-loop Agent

Role:

Provides a checkpoint control before execution continues.

Why required:

- Prevents hallucinated scripts
- Ensures user intent alignment

6.4 Character Designer Agent

Role:

Extracts and formalizes character identities.

Outputs:

- Name
- Personality traits
- Appearance description
- Reference style

Key Feature:

- Maintains **identity consistency across scenes**

MCP Tools:

- *commit_memory*
- *query_stock_footage*

6.5 Image Synthesizer Agent

Role:

Generates visual representations of characters.

Implementation:

- Stable Diffusion / ComfyUI (local)
- Accessed via MCP

Output:

- Character reference images

■ LangGraph Workflow Design

Nodes:

- *Mode_selector_node*
- *Validator_node*
- *Scriptwriter_node*
- *Hitl_node*
- *Character_node*
- *Image_node*
- *Memory_commit_node*

Script Representation

```JSON
{
  "input_mode": "manual | auto",
  "script": {},
  "characters": [],
  "images": [],
  "status": "processing"
}
```

■ Identity Handling

In this phase, early identity awareness begins:

- Characters are linked to:
  - Visual references
  - Metadata

This supports later:

- Face swapping
- Identity validation
- Anonymization

■ Deliverables

- *Scene_manifest.json*
- *Character_db.json*

- *Images/*
- LangGraph workflow
- MCP tool integration

■ Evaluation Rubric (Phase 1)

Criteria	**Description**	**Marks**
Agent Definition	Clear roles, reasoning loops	20
Script Generation Quality	Structured + coherent scenes	15
MCP Integration	Proper tool usage (no hardcoding)	15
LangGraph Workflow	StateGraph correctness	10
Human-in-the-Loop	Proper checkpoint design	10
Output Completeness	JSON + images generated	5
Total		**75**